Video transcript — MARCO Crystallisation Triage (verbatim, from the actual recordings)

This is no longer a pre-recording script — it's a **verbatim transcript** of the 9 raw clips actually recorded (`videos_raw/*.mov`), produced with local ASR (faster-whisper, `small` model, CPU). Timestamps are `[M:SS]` within each clip. This document exists so the real recorded content can be reviewed and trimmed deliberately in conversation, rather than cut automatically.

ASR accuracy note: language detection confidence was 0.99-1.00 (English) on every clip, so the transcription is generally reliable, but automated speech recognition still mishears domain-specific jargon and proper nouns. Known likely mishearings spotted on review (the transcript below is left as ASR produced it, not hand-corrected): - "MARCO" (the dataset) → transcribed as "Markov" / "Marko" throughout - "well" (as in plate well) → transcribed as "bell" in clips 2-3 - "HOG" (the feature type) → transcribed as "HRG" in clip 4 - "overfitting" → transcribed as "off-beat" in clip 5 - "unfreezing" → transcribed as "entrees in" / "entrose" in clips 5-6 - A few other small mishearings are left in place; none change the meaning seriously, but don't trust exact wording in those spots.

---

1. Intro (0:53)

> *[0:00] Hi, I'm Chanika Subashini. [0:03] This is my MSc Data Science coursework project. [0:08] I build a model that sorts protein crystallization drop [0:12] images into four classes, clear, crystal, precipitate, [0:18] and other using the Markov data set. [0:22] The A is to help a crystallographer [0:26] to decide which drops to look at first. [0:30] Only drop-set crystals are worth harvesting [0:34] and sending to a synchrotron beam line. [0:37] So for this user, missing a real crystal [0:42] is much more costly than a false alarm. [0:46] That imbalance guided every modeling decision [0:50] here I made in this analysis.*

---

2. Data download (1:32)

> *[0:01] The full macro dataset is large, about 463,000 images, around 55 gigabytes, stored as TF [0:11] record files split into shards. [0:14] The training part alone has 407 shards of about 1,000 images each. [0:22] To understand the data, I first downloaded one shard and counted how many images were [0:29] in each of the four classes. Then I wanted to know, do all the shards have a similar class mix? [0:36] So, I downloaded two more shards from different parts of the dataset and compared them. [0:45] The share of crystal images was close in all three shards 11.8, 13.2 and 14.8 percent. [0:54] The full dataset has 12.8 percent. One shard alone can be a little*

> *off, [1:04] but together the three shards matched the full dataset well. [1:10] This told me that a sample made from several shards would represent the whole dataset. [1:17] So, I used 15 shards, about 14,400 images for this project. [1:25] This is the script that downloads these 15 shards and extract the images.*

---

3. EDA (2:30)

> *[0:02] Next, the exploratory data analysis, leakage control. [0:09] The same bell is often photographed more than once, and a bell can hold several drops. [0:17] If those images ended up in both training and testing, the test score would look better [0:24] than it really is. [0:26] So, I read the plate and bell numbers from the file names and gave every image a key group key. [0:38] Images from the same plate and same well share a key. [0:43] Splitting the data. [0:47] Then I split the data into training, validation and test sets. [0:53] Roughly 70, 15 and 15 percent. [0:57] Using scikit-learn stratified group k-fold. [1:04] Group means whole groups stay together. [1:09] So, now well crosses a split. [1:12] Stratified means each set keeps a similar class balance. [1:19] I saved the split as a CSV file with each images ID, label, group key and split. [1:30] Every model uses the same file and I can check at any time that no group crosses the split. [1:39] Is the sample representative? [1:47] On the training and validation images, I ran a chi-square test. [1:56] Against the class counts of the full Markov dataset. [2:01] There was no significant difference. [2:04] So, my sample reflects the population. [2:09] Exploring the data. [2:11] I plotted the class distributions, the source institutions and the image sizes for each class. [2:21] Image sizes vary widely. [2:24] Which is why every image is resized before training.*

---

4. Classical model (1:52)

> *[0:00] Classical baseline. Before any neural network, I built a classical model to compare [0:07] our guest. Each image is turned into handcrafted features, HRG which [0:17] describes edges and shapes, and LBP which describes texture. A light GBM model [0:26] then learns to classify those features. I chose light GBM over XGBoost and SVM [0:34] because it trains quickly on almost 2,000 features and handles class in [0:42] balance well. I used balanced class weights so the reactryster class counts as [0:49] much as the others. Set in the precision role. On the validation set, I [0:59] plotted the baseline's precision recall curve for crystals. From it, I set my rule [1:07] at least 30% of the drops the model flags must really contain crystals. That [1:15] means out of every 10 flag drops at least 3 are real crystal drops. I fixed this [1:24] level before training any neural network and used it unchanged for [1:28] every model so it can't favor any of them. The result on the test set the [1:38] baseline found 93.7% of the crystal drops at this precision. It trained under [1:47] 30 seconds so it's a strong and fast model to beat.*

---

5. Initial ANN (3:01)

> *[0:00] Initial ANN [0:03] For the neural network, I used a ResNet backbone pertained on ImageNet. [0:09] I kept the backbone frozen and trained only a new head on top. [0:15] A dropout layer and a final layer for my four classes. [0:22] Hyperparameter search [0:26] I searched the three required hyperparameters [0:30] Learning rate from 0.0003 to 0.01, batch size 16, 32 or 64 and network depth ResNet 18 or ResNet 34. [0:45] That's 24 possible combinations. [0:48] I ran a random search of 12 configurations, each trained 12 epochs. [0:54] Because each setting was drawn independently, one combination came up twice. [1:01] So the search covered 11 different combinations. [1:07] Where I trained? [1:09] Training on my laptop was too slow. [1:13] So I needed a GPU. [1:15] I started on Google Collapse Free Tier. [1:19] But it kept cutting me off part way through the search. [1:22] A paid subscription was an option. [1:26] But I only needed a few hours of GPU time. [1:30] So renting a GPU on run pod by the hour was cheaper and more reliable. [1:37] The training code is exactly the same either way. [1:41] Only the machine changed. [1:44] What learning curve showed? [1:50] Training and validation loss both dropped in the first epoch. [1:57] Then flattened out and stayed close together. [2:03] So the model wasn't off-beat. [2:06] But it also stopped improving. [2:09] With backbone pros and only about 2000 numbers in the head can learn. [2:15] And then sit on top of features that were never adapted to drop images. [2:21] That plateau is what led to my next step. [2:26] The final A&N. [2:28] A failed experiment. [2:32] One setting failed. [2:37] The highest learning rate 0.01 with the smallest batch size 16. [2:43] Training became unstable. [2:45] The loss went up instead of down. [2:48] And crystal recall dropped to 0.62. [2:52] I kept it in my log rather than hiding it. [2:56] It showed that this model needs a smaller learning rate.*

---

6. Final ANN (2:16)

> *[0:00] final ANNIN. The plateau told me the frozen backbone was the limit. So I added a fourth [0:08] hyperparameter, degree of fine-tuning. I compared keeping the backbone frozen with [0:16] entrees in its last block layer 4 so the model could adapt its top-level features [0:23] to crystallization images instead of reusing image net features as they are. [0:31] You can see the effect in those two charts across the same dual configurations. [0:38] The final ANNIN scores higher than the initial ANNIN and its training loss keeps falling [0:47] across 12 epochs instead of flattening after the first because it now has room to learn. [0:56] Test results. The trivial benchmark was always guesses the most common class. [1:08] It gets 48% accuracy but finds no crystals at all which shows why accuracy alone is a poor measure [1:19] here. The classical baseline finds 93.7% of crystal drops. The frozen initial ANN [1:31] finds 88.6%. Interestingly it beats the baseline on validation but lost to it on the test. [1:42] The final ANNIN with layer 4 and frozen finds 96.5%. The best result. [1:53] The test score matched its validation score so the improvement is real, [2:00] not just tuned to*

> *the validation set. So my pre-registered expectation that the ANN would [2:08] win was correct but only after I entrose part of the backbone.*

---

7. XAI (2:56)

> *[0:00] This is the web app. Fast API backend, simple frontend. [0:06] Every prediction here runs the actual trained model fresh right now and nothing is pre-computed. [0:16] Let's upload a crystal image and get the predictions. [0:25] This heatmap is gradcam. It shows which part of the image most influenced the prediction. [0:38] I compared it with two other methods. [0:42] Occlusion covers small patches of the image one at a time and measures how much the prediction changes. [0:54] Line split the images into regions and tests which one matters most. [1:02] Gradcam is the only one fast enough to run live so it is the one in the app. [1:10] The other two I used offline to check it. [1:14] For crystallographer, gradcam shows where to look. It doesn't show how the model raises. [1:22] I checked that it is trustworthy in two ways. [1:32] First, when I covered the highlighted area, the model's crystals fell from 0.62 to 0.14. [1:44] So that area really drives the prediction. [1:50] Second, when I reset the model's trained layers to random weights, the heatmap changed completely. [2:00] So it reflects what the model learned. [2:04] Let's find a real mistake. [2:16] This drop really contains crystals but the model gives it only a 0.8% chance of crystals. [2:34] That's the costly error. A missed crystal. [2:40] All three methods points at the center of the drop. [2:49] So the model is looking in the right place but reading the texture wrong.*

---

8. GenAI (0:28)

> *[0:00] Finally, I compared all of this against a pre-trained vision language model Claude Haiku 4.5, zero shot, [0:11] not training on Marko at all, evaluated on the exact same test images with the same metric. [0:20] It did worse than every trained model and worse than the trivial majority class benchmark.*

---

9. End (0:51)

> *[0:00] To sum up, I built a fixed split that keep images from the same well together. [0:08] So there's no leakage. [0:10] A classical baseline set a high bar. [0:14] The neural network beat it only after I unfroze part of the backbone. [0:21] Gradcam shows where the model looks and I check that it's trustworthy. [0:28] And a general AI model with no training on this task did far worse, which shows why training [0:37] on real crystallization data still matters. [0:42] The prediction in the app ran live with nothing pre-computed. [0:50] Thank you for watching.*

Running time vs. the 8:00 target

Clip	Duration	Running total
1. Intro	0:53	0:53
2. Data download	1:32	2:26
3. EDA	2:30	4:55
4. Classical model	1:52	6:48
5. Initial ANN	3:01	9:49
6. Final ANN	2:16	12:04
7. XAI	2:56	15:01
8. GenAI	0:28	15:29
9. End	0:51	16:20

Total raw runtime: 16:20 (979.8s). Target: 8:00 (480s). Gap: ~8:20 needs to come out.

No cutting decisions are made in this document - this is the record of what was actually said, for review before deciding together what to trim.